



NOVIA: A Framework for Discovering Non-Conventional Inline Accelerators

David Trilla
IBM Research
Yorktown Heights, USA

John-David Wellman
IBM Research
Yorktown Heights, USA

Alper Buyuktosunoglu
IBM Research
Yorktown Heights, USA

Pradip Bose
IBM Research
Yorktown Heights, USA

ABSTRACT

Accelerators provide an increasingly valuable source of performance in modern computing systems. In most cases, accelerators are implemented as stand-alone, offload engines to which the processor can send large computation tasks. For many edge devices, as performance needs increase accelerators become essential, but the tight constraints on these devices limit the extent to which offload engines can be incorporated. An alternative is inline accelerators, which can be integrated as part of the core and provide performance with much smaller start-up times and area overheads. While inline accelerators allow greater flexibility in the interface and acceleration of finer grain code, determining good inline candidate accelerators is non-trivial. In this paper, we present NOVIA, a framework to derive inline accelerators by examining the workload source code and identifying inline accelerator candidates that provide benefits across many different regions of the workload. These NOVIA-derived accelerators are then integrated into an embedded core. For this core, NOVIA produces inline accelerators that improve the performance of various benchmark suites like EEMBC Autobench 2.0 and Mediabench by 1.37x with only a 3% core area increase.

CCS CONCEPTS

• **Hardware**; • **Computer systems organization** → **Architectures**;

KEYWORDS

inline accelerator, hardware-software co-design, accelerator discovery

ACM Reference Format:

David Trilla, John-David Wellman, Alper Buyuktosunoglu, and Pradip Bose. 2021. NOVIA: A Framework for Discovering Non-Conventional Inline Accelerators. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21)*, October 18–22, 2021, Virtual Event, Greece. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3466752.3480094>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO '21, October 18–22, 2021, Virtual Event, Greece

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8557-2/21/10...\$15.00

<https://doi.org/10.1145/3466752.3480094>

1 INTRODUCTION

Most domain-specific, focused edge devices utilize smaller, simpler, lower-area cores, which provide only a modest performance. To meet their area-performance goals, these systems usually supplement the cores with a set of domain-specific offload accelerators. Offload accelerators are typically attached across a system interconnect (e.g., as a virtual IO device) and operate from a local memory space which must be loaded prior to invocation, and read out at finish. While offload accelerators provide significant speedups for portions of the workload, an offload accelerator can generally only be developed for sections of the workload that provide significant per-invocation execution to cover the invocation overhead.

While the addition of offload accelerators can provide large speedups for many edge workloads, this tends to still leave significant execution time in the non-accelerated portions of the workload, which must run on the modest-performance cores. This non-accelerated code can, as shown by Amdahl's Law, become a significant limiter on overall workload efficiency. In order to improve the workload efficiency further, one must consider improving the execution of the remaining workload code running on the cores. Our solution is inline accelerators, which are implemented in the core itself, much like custom instructions or an additional function unit. Inline accelerators use core pipeline interfaces and are not managed by system software, giving them much smaller start-up times, and allowing them to provide speedup on smaller pieces of the workload. Notably, inline accelerators can be used to try to speed up areas that offload accelerators cannot cover, and can be applied across many different parts of the workload (essentially by accelerating “common subexpression” terms across the code run on the core).

While inline accelerators can fill many of the gaps left by offload acceleration, there are also several real challenges to identifying and implementing inline accelerators. Inline accelerators can embody very small computations and be applied anywhere in the workload. This means that the identification of inline accelerators must consider a large body of workload code, and look for small (common) sequences of operations across the entire workload domain to identify good candidates. Given the overall design limits on area, the total number and size of the inline accelerators must also be limited, and thus the search process must analyze the overall benefit (across the workload) across many such candidates, and strive to select only those design candidates that produce the greatest performance benefits for the least overheads.

Further, the implementation of inline accelerators requires not only the accelerator design (e.g., the functional RTL) but also the interface within the core. To minimize the development costs, one must be cognizant of core re-design and re-verification costs as well. Finally, because inline accelerators typically present as new instructions, they can also create future ISA legacy and drive software stack pressure (e.g., in the compilation framework to support the use of these new accelerator instructions).

Because of the many challenges to developing and deploying inline accelerators, in this paper we propose NOVIA: a framework to automatically search the design space of a given workload and to identify and derive inline accelerator candidates that optimize the area and performance trade-offs by exploiting the common execution patterns throughout the workload. NOVIA transforms important code sequences through a unique merge and split methodology into shared work subgraphs, and extracts the reducible computations. Then, guided by the area and performance impacts, NOVIA extracts the best candidates, and provides implementations of them as inline accelerators.

In particular, NOVIA provides several contributions to enable the design and deployment of inline accelerators, including:

- (1) An automatic search over general, complex workload space for common inline accelerator candidates.
- (2) An efficient merging methodology to combine separate execution paths into composite realizations (saving area).
- (3) An efficient splitting methodology to extract final accelerator functionality from a merged candidate (ensuring interface compliance).
- (4) Analysis and ranking of the performance benefit and area costs for each candidate.
- (5) A general inline accelerator interface for the core to support any number of inline accelerators (reducing core re-design and re-verification costs).
- (6) A sensitivity analysis over different initial conditions.

Overall, to the best of our knowledge, no prior art tackles the full problem of efficient inline acceleration at arbitrary granularity with an end-to-end methodology. NOVIA provides an end-to-end framework to derive area efficient inline accelerators that profiles the workload and applies a unique merge/split process to coalesce common sub-graphs, and generates the accelerator design.

2 MAIN NOVIA CONCEPTS

2.1 The Instruction Sequence

At its core, our methodology works to identify sequences of workload operations (i.e. instructions) that can be compatibly combined together. The basic unit of optimization, therefore, is a sequence of these instructions, and we refer to this as an Instruction Sequence (IS). An IS can range from a single instruction, to a basic block (BB), or even many instructions from multiple blocks. The choice of granularity has a number of impacts on both the execution time, and the final extracted accelerator candidates. Using a higher-granularity IS generally results in larger accelerators with commensurate memory footprints and resource requirements.

Our methodology represents the IS in a directed acyclic Data-Flow Graph (DFG). Figure 1(③) illustrates a simple illustrative example of a DFG formed by merging the initial IS from the code

in Figure 1(①). In this example, IS1 (outlined in blue), IS2 (in orange), and IS3 (in yellow) have been combined to form the single, merged IS of Figure 1(③).

2.2 Functionality Merging

We refer to the correct operations, ordering, data manipulation and output of a given IS as its Functionality. NOVIA essentially *combines* and *reduces*, through merge and split processes, the functionalities of multiple IS into a single merged IS that contains all the component functionalities. The basic fundamental of merging is that the execution of the merged IS can produce the results of any one of the original IS functionalities, provided that the correct inputs are given. The merged IS, however, can realize this set of functionalities in less total area because the merged realization uses the same components to cover common operations and sequences across several of the original independent IS paths.

A Merged IS contains multiple simultaneous paths that share internal logic, input and output values. Therefore, selection operations (e.g. muxes) are added in order to preserve the original DFG functionalities. These selection operations are controlled by external inputs, much like the function selection for an ALU.

2.3 Extracted Inline Accelerator

The NOVIA methodology will ultimately produce a merged IS, which can be directly used as the definition of an accelerator, or can be further simplified by reducing the content of the accelerator definition using an IS-splitting process. In this paper, we focus on inline accelerator units that are resident in the core, implemented as specialized functional units (NOVIA FUs or NFUs) driven by custom instructions. Focusing on an NFU type of inline accelerator provides for a minimal invocation overhead and a uniform, reusable accelerator interface within the core design (see Section 5). The resulting accelerators will generally be relatively small, fed by results computed in close proximity, have few inputs and outputs, and not require local scratch-pad memories. An example of this can be seen in Figure 1 (④), where two different subgraphs have been extracted (split) from the originally-merged IS (③).

An inline accelerator's performance is determined by the longest-latency path (functionality) from the merged IS blocks, i.e. the latency of the longest merged critical path. It is important, therefore, not to merge independent long-latency and short-latency paths into a single shared accelerator in hopes of providing maximal speedup for the shortest paths. This leads to a three-part set of guiding principles for the NOVIA merging analysis:

- (1) When merging two IS blocks, the critical path is monotonically non-decreasing.
- (2) When two non-isomorphic DFGs are merged, additional control logic (selects) must be added to preserve the correct functionality, increasing the delay for that IS path.
- (3) Any such increased IS delay must not meet or exceed the critical path delay of the original main computation in order for the merge to be considered beneficial.

Summarizing, merging IS to form accelerators presents a design optimization trade-off. Increasingly merging IS saves area, but decreases some performance potential by increasing the critical-path

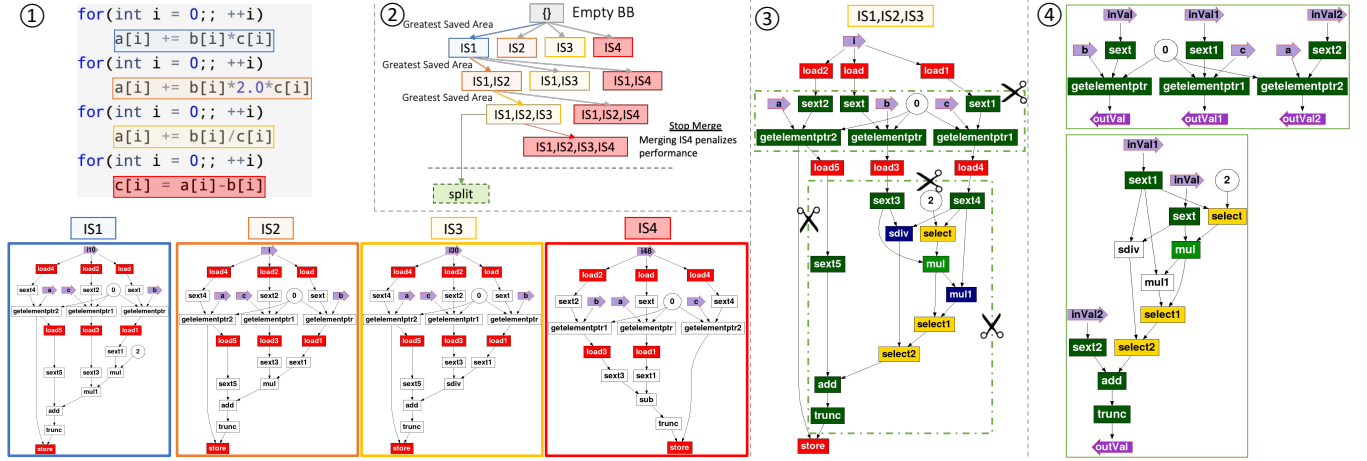


Figure 1: Starting from source code ① and a list of profiled IS, the DFGs of each IS are derived. The merge process ② combines IS, and selects (and prunes) the merged IS carried forward. In this example, IS1, IS2 and IS3 are merged together while IS4 is excluded. The resulting final merge candidate ③ is passed to the split stage, which generates the final accelerator candidate ④ for realization. DFGs ③ and ④ show live-ins as right arrows, live-outs as left arrows, merged instructions in green, memory in red, multiplexer in yellow, and non-merged instructions in blue or white.

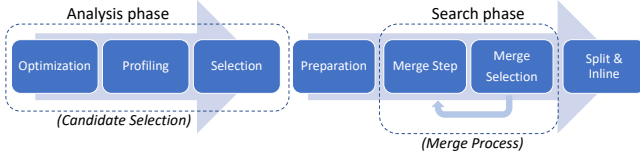


Figure 2: Methodology flow to derive inline accelerators.

(i.e. run time for all functionalities). Simultaneously, using merged accelerators rather than individual functionality accelerators will save area, and the area savings provides space for additional accelerator implementations, therefore increasing performance.

3 NOVIA METHODOLOGY

The NOVIA methodology has four main steps in three processes:

Preparation: The first step prepares the workload and determines the set of IS Candidates that constitute the pool of IS to be used in the subsequent accelerator search analysis. One of these is denoted the Base IS, which is used as the starting point for subsequent accelerator analysis.

Merge Process: The *Merge* and *Merge Selection* steps.

Merge Step: The methodology considers pairwise merges of the Base IS candidate with each remaining member of the IS Candidate Pool. The merge step combines the two IS DFGs into a single, unified DFG that can realize, based on input controls, either of the two original IS functionalities.

Merge Selection: After each merge step, each newly-merged IS is then evaluated for various benefit metrics (e.g., area, performance). If a new merged IS improves the metrics, then the highest-scoring merged IS is retained as the new Base IS for subsequent merge steps. If no new merged IS shows improvement, then the previous Base IS is retained, and a new Base IS is selected from the remaining pool of unmerged IS. This process of Merge Step followed by Merge IS Selection continues until the new merges cease to provide benefit.

Split & Inline: In this final step, after all merging is done, the merged IS are then split to find the most beneficial components (i.e. subgraphs) that provide the greatest commonality across the individual functionalities (i.e. most re-use) and then each subgraph is used to generate an inline accelerator.

3.1 Preparation

Preparation is the first step in the NOVIA methodology. It involves selecting and preparing within an initial IS Candidate Pool from a representative set of workloads, a starting Base IS. There are many ways to provide the initial IS Candidate Pool and Base IS; experts could determine which are the IS of interests, or an automatic process can be employed using some measurement criteria. To tackle this problem automatically, one viable way is to extract the initial candidates by means of a profiling analysis using representative inputs. The profiling will identify the sections of the code that are significant contributors to overall performance, and can rank these from highest to lowest to provide a sorted list of IS Candidates, with the highest also being set as the starting Base IS.

As the accelerator merge process iterates through pairwise merges of the Base IS with all remaining Candidate IS, the size of the Candidate IS Pool will significantly affect the execution time of the merge search. Without loss of generality, one can reduce the Candidate IS pool to a subset of the (highest impacting) N IS candidates.

3.2 Merge Process

The merge process, is a guided methodology, following a greedy algorithm, that searches for accelerator candidates by merging Candidate IS, and pruning those merges that are sub-optimal or infeasible. This merge process is divided into 2 main components, the **merge step** and the **guided merge selection step**, described below.

3.2.1 Merge Step. The **merge step** is at the core of the methodology. This step takes two IS as inputs and generates a unified (i.e.,

merged) IS that, given the appropriate input parameters, is able to execute either of the merged IS functionalities. Figure 1(①②③) shows a simple example of how the merge process works.

The merge algorithm joins two different IS graphs into a single one, while reusing as many operations as possible. Each merged operation results in area savings, and the performance is improved by the accelerator hardware exploiting the Instruction Level Parallelism (ILP) of the IS and the performance advantages of combinational versus sequential logic. In general, the merge operation is similar to checking for graph isomorphism, though the complexity of graph isomorphism analysis can make the problem intractable. To avoid this complexity, NOVA applies a greedy algorithm to infer which operations to merge. Therefore NOVA is not necessarily finding the most optimal path merge (i.e., the longest common sequence of operations) but rather a near-optimal result using simpler merging of identical individual operations along the paths.

To preserve functional correctness, and ensure that the specific functionality of the merged IS is performed correctly, data must be correctly applied across the DFG. This is ensured by applying certain restrictions to the merge operation. First, an operand to a merged operation may come from one of two different paths, requiring a *select* operation (a *mux*). Trees of these *select* operations (e.g. after several paths merge through the same operation) can usually be collapsed, especially when merged operations feed other merged operations. Secondly, two operations will never be considered for a merge if that merge would create a cyclic path in the DFG.

Algorithm 1 describes the algorithm used in the **merge step** process. Given an IS_a consisting of an original (i.e. unmodified) IS and IS_b , a merged IS¹, we iterate across the instructions of the original IS, and we search in our merged IS to identify a mergeable instruction (sharing the same opcode). If found, we then check that adding the edges from the original IS into the merged DFG will not create a cyclic path in the merged graph. Once the instruction is merged, we continue to analyze whether the operands can also be merged. For each unmerged operand, we add *select* instructions to provide the correct operand routing. The end result is illustrated in the DFG of Figure 1(③), where the green color gradient indicates the number of merges a specific operation (node) represents across the different merged IS. Note that, in this specific case, only one instruction (*mul*) was multiply merged, so we observe only two shades of green.

3.2.2 Guided Merge-Selection Step. The **merge step** performs a *cumulative* merge of two ISs, but because NOVA employs a greedy algorithm on the **merge step**, the order in which IS is merged affects the final resulting merged DFG. An optimized search algorithm must therefore be applied to sort out good merges from bad merges, and to make a final selection on which merge candidates are to be selected/retained.

To that end, we follow a selection methodology similar to a tree traversal and pruning search, which allows a balance of exploration and exploitation by testing some initial merge combinations, selecting the most optimal, and iterating on the process. The guided selection process starts with an empty IS, which is successively merged with all the initial available IS candidates into new merged

Algorithm 1: Merge Step Algorithm

```

Input:  $IS_a$  and  $IS_b$ ; Init:  $list_{merged} \leftarrow \emptyset$ ;
for  $i \in IS_a^{ins}$  do
  for  $j \in IS_b^{ins}$  do
     $merge \leftarrow \text{checkInstMergeable}(i, j)$ ;
     $noloop \leftarrow \text{checkNoLoop}(i, j)$ ;
    if  $merge \wedge noloop \wedge j \notin list_{merged}$  then
       $list_{merged} \leftarrow i; list_{merged} \leftarrow j$ ;
      foreach operand pair  $op_i, op_j$  do
        if  $\neg \text{checkOpsMergeable}(op_i, op_j)$  then
           $AddSelect$ ;
        break;
    if  $i \notin list_{merged}$  then
       $IS_b^{ops} \leftarrow copy(i); list_{merged} \leftarrow i$ ;

```

candidates. These merged candidates are then evaluated by various models, deriving scores for various merit figures (e.g., area, critical path, speedup). Once each combination has been evaluated, a selector function chooses from among the candidates' measurements the best current merge candidate to retain as the new Base IS.

All non-selected merges are then discarded, the candidates that were merged are removed from the candidate IS list, and a new search iteration begins with the new Base IS. This time, the remaining unmerged IS candidates are merged with the previously selected best merged IS, and the selection process is repeated across those merges. If no beneficial merges are selected in a round, then the current merged candidate is saved, and the merge process begins with a new Base IS selected from the remaining initial candidate IS. This process continues until there are no more initial candidates to merge. Algorithm 2 and Figure 1(②) depict how the selection process drives candidate selection and explores the merge space.

Algorithm 2: Guided Merge-Selection Step

```

Input:  $list_{candidates}$  Initial Candidates;
Init:  $list_{inline} \leftarrow \emptyset$ ;  $candidate \leftarrow \emptyset$ ;
while  $list_{candidates}$  not empty do
  for  $i \in list_{candidates}$  do
     $list_{merged} \leftarrow \text{mergeStep}(i, candidate)$ ;
   $improves \leftarrow \text{evaluate}(list_{merged})$ ;
   $sort(list_{merged})$ ;
   $max \leftarrow list_{merged}[0]$ ;
   $Init_{candidate} \leftarrow \text{getInitialCandidate}(max)$ ;
  if  $improves$  then
     $candidate \leftarrow max$ ;
  else
     $list_{inline} \leftarrow candidate$ ;  $candidate \leftarrow \emptyset$ ;
  remove  $Init_{candidate}$  from  $list_{candidates}$ ;

```

Evaluation Functions: The evaluation functions are a set of merit figures, models and evaluation parameters that allow us to

¹Note that the merged IS can be empty, hence we can derive any individual candidate by merging it with an empty IS.

rank the merged IS according to different scores. Our methodology allows for those functions to be user-defined and therefore use different search patterns during our guided merge selection step. In this work, we have focused on two main merit figures: saved time (S_t) described by Equation 3, and saved area (S_a) of the merged IS over the independent IS as accelerators, described by Equation 2.

To derive our measurements, we first compute the individual figures for each initial IS (IS_a, IS_b), and then for each merged IS (IS_{ab}). To determine the area estimate (A_{is}), the model adds together the individual per-operation area values to compute the total area of an IS, were it realized in hardware.

To model the performance, two different measurements are derived for each IS. The first of these is the sequential time ($Tseq_{is}$), which is the sum of all the individual operation execution times. Thus, ($Tseq_{is}$) is representative of the time required to execute each operation of the graph sequentially, and of the single-issue embedded core's execution time. In contrast, the critical path delay (Cp_{is}) is measured as the time required to execute the critical path (i.e. longest path through the DFG) operations sequentially; this corresponds to the case where all parallel DFG operations are executed in parallel, and the execution time is determined by the longest-delay sequential path. The difference of these two values expresses the potential performance difference (*speedup*) between the sequential and hardware-accelerated versions of the IS.

Equation 2 derives the amount of area saved by merging two IS, where $A_{is}^{original/merged}$ is the area of an IS (original or merged) as defined by Equation 1 as the addition of the area of all its individual operations. Whenever two merged IS share an operation in the merged graph, the cumulative area for the merged IS is smaller by one copy of that operation. Essentially, the saved area is the area saved by merging the IS functionality paths as opposed to implementing a separate accelerator for each functionality.

$$A_{is} = \sum_{i=1}^{N_{instr}} A_i \quad (1)$$

$$S_a = A^{merged} - \sum_{i=1}^{N_{is}} A_i^{original} \quad (2)$$

$$S_t = \sum_{i=1}^{N_{is}} Tseq_i * Iter_i - Cp_{is} * \sum_{i=1}^{N_{is}} Iter_i \quad (3)$$

At each *select* iteration, after all the new merge candidates have been evaluated, the merged IS are sorted by saved area. If the amount of saved area has increased, then the merging search continues, using this merged IS as the Base IS for further merging, as shown in Figure 1(2). If, however, the amount of saved area has decreased with respect to the previous iteration (i.e., the previously selected merged IS saved more area than the current candidate) then the merge process is stopped, as further merges are likely not beneficial, and no merged candidate is selected. At this point, the current best merged IS is set aside (retained), and the merge process starts anew for the remaining, as yet unmerged IS Candidates.

Throughout the merge process, the merged IS area and performance must constantly be traded off. Merges create a larger and usually deeper IS, generally increasing the critical path delay and thereby decreasing inline speed-ups. To factor in that effect, we

make use of Equation 3, which represents the amount of time saved by realizing the IS in hardware, where $Tseq_i$ is the sequential time of executing IS_i , $Iter_i$ is the number of *iterations* or times that IS_i is executed, and Cp_{is} is the time (critical path) to execute the merged IS. Hence all candidate merges must ensure the result of Equation 3 is positive to ensure that the increased critical path from merging larger IS still provides benefit. Hence, if the saved time equation is negative, then the merge candidate is discarded.

3.3 Split and Inlining Process

The Split and Inlining process is the last step of the NOVIA merge process. At this stage, the greedy merge algorithm has determined a set of quasi-optimal merges, having merged the most beneficial candidates and discarding those that do not offer significant benefits. While each merged IS has already been deemed relatively optimal, these merged IS are required to be “complete” in the sense that they fully embody the execution of the original IS from which they were composed. In many cases, this results in the merged IS including branches in the DFG that were contributed by a single original IS, have minimal contribution to the overall speedup, but increase the overall area required to realize that accelerator.

As our target in this work is small, highly efficient inline accelerators accessed as instructions (and therefore having relatively few live-in and live-out connections), we optimize the selection of the inline accelerator from this full merged candidate. NOVIA applies a “split” step that reduces the final merged IS to a (set of) minimal expression DFGs. This pruning step removes those parts of the DFG, such as unmerged operations that provide little speedup but cost area in an inline accelerator, as well as unwanted elements like memory-access operations.

Practically, the split operation scans the merged IS DFG and breaks it into subgraphs by considering memory instructions as the new graph boundaries. The split process then considers the root nodes of these new subgraphs, and searches for *select* operations (inserted during merge to guide the input for different merged functionalities) that are now exposed as live-in values. These select operations can be removed as the live-ins are now accelerator input parameters. A simple example of this split processing is illustrated in Figure 1(3). The split is shown by the dashed green line with scissors, symbolizing subgraphs that extract the most-overlapped computation segment, and separating the computation from the memory/interface boundaries. The split on the bottom results in an inline accelerator that takes three input values and produces a single output value (i.e., three live-ins and one live-out), as seen in Figure 1(4).

3.4 Summary

To summarize, there are three main processes:

Preparation: In the preparation step, a set of Instruction Sequences (ISs) of the workload are extracted and prepared to serve as the Candidate IS pool and a Base IS is chosen.

Merge Process: In the Merging step, the candidate ISs are systematically merged (pairwise) to identify and Select the most optimal combinations. Merging continues from one Base IS until further merges result in no benefit, at which point the Merging process is applied to the remaining candidate ISs to find additional accelerator candidates.

Split & Inlining: In the Splitting & Inlining process, the accelerator candidates from the Merging step are considered (individually), and the highest-benefit portions of those candidates are extracted for implementation as simple and small inline accelerators. Note that for other targets (e.g., complex accelerators) this splitting step may not be needed. Finally, we also perform the Inlining, where the extracted accelerators from the Splitting step are analyzed and converted into hardware, and interfaced with the core.

4 NOVIA FRAMEWORK

NOVIA is primarily implemented as a series of LLVM [41] passes that analyze and modify the LLVM IR representation. The entire process is self contained, providing instrumentation, code generation and accelerator definition. Starting from the source code, NOVIA generates a compilable, executable LLVM bytecode incorporating a set of extracted functions that define and execute the selected accelerator candidates.

The framework implements two phases in several steps, as shown in Figure 2. The first is an analysis phase, where code optimizations and priming take place, followed by a search phase, where NOVIA applies a Merging process which produces the optimized set of combined IS accelerator candidates. Finally, the framework applies a Splitting process to extract from those candidates the most promising, realizable inline accelerator candidates.

4.1 IS Realization

In this paper, NOVIA uses a basic block (BB) as the initial IS and follows a bottom-up approach from these BBs to merge into a set of larger optimized IS, then splitting from those the final accelerators. NOVIA, however, is not dependent on the specific IS granularity used here, and could operate at a higher level of granularity and identify more complex accelerators (e.g., starting with functions as the basic IS). Here we focus on the simpler IS and inline accelerators as in Section 2 to facilitate automatic generation of the accelerators and their integration and use as inline in-core function units.

4.2 Analysis Phase

NOVIA starts with an analysis phase, which uses LLVM transforms to prepare the workload for further NOVIA processing. This phase involves three distinct stages: workload *optimization*, *profiling*, and *selection*.

Optimization: In the optimization stage, the workload is transformed from source code into a NOVIA bytecode file. The first step is compilation, using the *wllvm* [51] compilation interface, which produces a single bytecode file containing the entire workload.

Three main transformations are then applied to this bytecode file. First, *renaming* ensures that each block and function name is unique. Next, the promotion (if possible) of memory operations to registers, enables longer streams of pure computation. Finally, *inlining* flattens the code into a single stream of instructions with no function calls.

Profiling: After Optimization, the bytecode is ready for Profiling, where NOVIA generates a second bytecode file that embeds the instrumentation required to produce per-BB execution counts and cycle counts. This instrumented bytecode file is compiled and run, producing the relative execution time each BB contributes to the overall workload time.

Selection: Once profiling information is available, NOVIA enters the *Selection* process. Given user-specified criteria, the framework sorts the BBs and returns the first N of these that match that criteria as the initial IS candidates. The simplest criterion is a set number N of initial BBs, but one could apply other criteria such as total execution-time coverage.

4.3 Search Phase

The Search Phase is also implemented as 4 distinct steps as a standalone LLVM Pass, which takes three inputs from the Analysis phase: the list of initial IS candidates (BBs), the profiling information, and the unified bytecode file of the workload.

Preparation: The Preparation process then identifies each BB in the bytecode file, and extracts it by (1) stripping branches, (2) marking live out values, and (3) relocating LLVM's *phi* instructions to other BBs.

Merge step & Merge Selection: Once all BBs have been prepared, the methodology of Section 3 executes the Merge Process, iterating through the BB candidates and merging until the exploration algorithm finishes. The result of the Merging process is a functionally correct, executable LLVM bytecode that contains a set of functions that embody merged-IS acceleration candidates. This executable is one means to validate that the workload functionality is unchanged, and this LLVM bytecode can be compiled into a working binary for further analysis or instrumentation.

Split & Inline: The final stage (*Split & Inline*) extracts, from each merged IS candidate, one or more inline accelerators by applying a guided splitting algorithm. This process, described in Section 3, can identify the “most common” (i.e., highest merged) sub-region of the merged DFG, and extracts it by cutting along a boundary that works to minimize the live-in and live-out interface while retaining a maximal accelerator benefit. In NOVIA, these resulting inline accelerator candidates are functionally correct implementations of the merged execution paths and are then passed to HLS tools for transformation into an RTL implementation, ready to be embedded in the core pipeline as an inline accelerator.

5 NOVIA FU CORE INTERFACE

In this work, an NFU (NOVIA Functional Unit) is implemented into the core pipeline as a functional-unit, with the core decoding and issuing NFU instructions. A typical ISA defines a number of instruction formats, but these are typically restricted in size and content, and often provide no more than 2 or 3 logical register name fields for operands. This necessarily limits the set of inputs and outputs available to an NFU. While NOVIA can restrict the selected NFUs to fit within these restrictions, we would like to realize more complex functionality. To this end, we have developed an ISA enhancement that provides for a larger set of inputs and outputs for the NFU, while requiring minimal alteration of the original core pipeline. The following subsections describe our NFU interface.

5.1 Hardware Support

Because we are targeting inline accelerators within the core pipeline, we have already accepted altering the core implementation, and any associated work that entails (e.g., re-generation of the physical

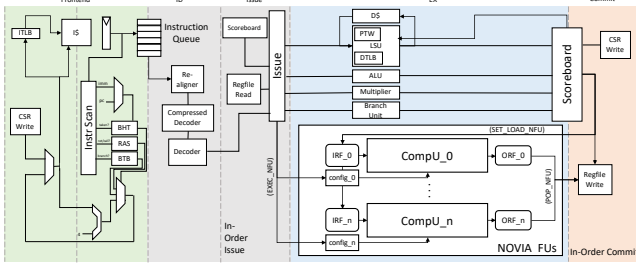


Figure 3: General architecture of an NFU embedded in a RISC-V Ariane core [6].

31	27	26	22	21	17	16	12	11	7	6	0
imm[4:0]	imm[4:0]	imm[4:0]	imm[4:0]	imm[4:0]	imm[4:0]	imm[4:0]	imm[4:0]	imm[4:0]	SET_LOAD_NFU		
RESERVED					imm[4:0]	imm[4:0]	imm[4:0]	EXEC_NFU			
RESERVED					rd	imm[4:0]	POP_NFU				
					funct		opcode				

Figure 4: Format of the NOVIA custom instructions

design, re-validation of portions of the core). However, we still attempt to produce a minimal impact to the original pipeline design, to limit the scope of any such work items.

Figure 3 depicts the high-level architecture, interface and integration features of an NFU, which illustrates the four elements that comprise an NFU implementation: an Input Register File (IRF), a Computation Unit (CompU), an Output Results FIFO (ORF), and a Configuration Control (Config) that embodies controls for the functionality to be realized by this NFU on this invocation.

The IRF stores the input operands that will drive the computation block of the NFU. These input registers must be loaded prior to the NFU execution, and each logical IRF register number corresponds to a particular input of the computational block (function), fixed by the physical wiring in the NFU design. Different functionalities of the NFU will therefore utilize different sets of IRF entries, but in a fixed fashion for that implemented NFU. Similarly, the results of the NFU computation are written into the ORF in a known order, so that the order of each output corresponds to a fixed output of that functionality. Each NFU can realize multiple different functions, each of which utilizes a subset of the input registers and produces a different set/order of the output results.

The NFU execution (start) is triggered by an instruction, similar to other functional units. Like an FXU (Fixed Point Unit), each NFU can represent several different functions (or execution paths) so each NFU can be activated by several different instructions. In actuality, these different instructions are grouped into a general NFU activation instruction where fields of the instruction indicate (a) the NFU to be targeted, and (b) the function on that NFU to activate during this execution.

The self-contained design of the NFU requires little modifications of the core: the NFU contains its Register files (IRF, ORF, Config), so essentially just the logic to load values into IRF, to receive the ORF register values, and to decode the NFU-interface instructions. In total, 175 lines of system Verilog were added to the existing Ariane code, and 454 lines were added here for the NFU module itself. These changes largely constitute a one-time design investment into the RISC-V core, as subsequent implementations

with different NFU contents can use the same interface implementation. For hardware overhead, the design of the NFU's controls (IRF, ORF, Config) consumes 476 LUTs and 1125 flip-flops when it is synthesized for a Xilinx VC707, and does not impact the critical paths.

5.2 ISA Support

The interface for a program to utilize an NFU is kept relatively simple and lightweight. The program must ensure that the inputs for the NFU are set, then activate the NFU function computation, and finally receive the output (result) values, and this interface is provided by a small set of custom instructions. Figure 4 shows the definition of our main-pipeline, NFU interface/control instructions.

Because the NFU uses the IRF to contain the live-in input values for the current computation, the appropriate IRF entries must be filled prior to NFU activation. One option would be to use a stream of "load-IRF" type instructions to send the operand values to the NFU and while this is functionally sound, it can result in unacceptable overheads.

In our approach, the **SET_LOAD_NFU** prefix instruction is used to indicate for a set of following instructions, (a) whether each instruction writes to the IRF, and if so, (b) which target IRF register. The (**SET_LOAD_NFU** imm5, imm5, imm5, imm5, imm5) instruction has five operands, each of them a 5-bit immediate value. The first immediate indicates which NFU is to be loaded, and the next four immediate fields indicate which input register (from 1 to 31) of that NFU should be written with each of the next four instruction's results. Note that a zero value in these IRF target fields indicates that instruction does **not** write into the NFU's IRF.

To illustrate, if we wanted to load NFU number 4, which has two live-in values that are provided by the two instructions following the **SET_LOAD_NFU** instruction, we would insert (**SET_LOAD_NFU** 4, 2, 5, 0, 0) before the first instruction generating a live-in value. In this example the loads will target NFU '4', with the first instruction following the prefix writing its result into register '2' of the IRF, and the next instruction into IRF register '5', while the next two, with values of '0' in the input mask, will not write into any IRF.

With this implementation, the number of allowable live-in values to any NFU is limited by the width of the immediate value fields in the instruction. Here, we have elected to use 5-bit fields allowing up to 32 NFUs and 31 registers per IRF, but these values could be adjusted as desired for a particular set of NFUs and registers per IRF. Note also that a single **SET_LOAD_NFU** is in this case only able to specify IRF targets for the next 4 instructions, but one can use several to cover a longer stream of instructions.

Once all values are loaded, a call to (**EXEC_NFU** imm5 a, imm5 b) indicates NFU *a* should start executing the configuration *b*. The immediate values here are used to index a particular NFU's configuration table and to set a specific pre-set configuration for the internal multiplexers to realize a particular functionality of that NFU.

Finally, once the function has completed, the NFU sends a finish indication to the core in a manner similar to other variable-length functional units. At finish, the NFU results are in the ORF, and it is necessary to bring the results back into main pipeline for use by the rest of the workload. In our implementation, we've elected to utilize the **POP_NFU** instruction, shown in Figure 4, which

specifies the NFU (in the *imm5* field) from which to pop the next output value, and the target main-core register (*rd*) into which that result should be written. We've elected the semantics of a FIFO stack because the NFU outputs are generated in a known, fixed order for each invocation configuration, and a FIFO stack provides a very simple interface. To illustrate the entire interface in practical use, we provide a simple example of the use of this interface in Listings 1, 2 and 3.

Listing 1 shows a simple NFU which takes 3 inputs (*i1*, *i2*, *i3*) and generates 2 outputs (i.e., *o1*, *o2*); *c0* is a constant, and *t0* is a temporary internal register value that does not have to be retained.

```
1  t0 = fmul i1, c0
2  o1 = fadd i2, i3
3  o2 = fmul t0, o1
```

Listing 1: Example Single-Function NFU

Listing 2 illustrates the instruction sequence leading to an invocation of the NFU of Listing 1. Here, instructions 2, 3, and 5 generate the inputs to the NFU in the IRF registers indicated in the comments. In order to correctly call the NFU, the code must be updated as in Listing 3.

```
1  r4 = fsub r17, r9
2  r3 = fmul r11, r21 # Input i3 of FU
3  r1 = fsub r7, r5   # Input i1 of FU
4  r9 = fadd r11, r8
5  r2 = fmul r13, r15 # Input i2 of FU
```

Listing 2: Instructions leading to accelerated code

The **SET_LOAD_NFU** instruction indicates that three of the following 4 instructions will load IRF registers, i.e. *r3* is written to IRF entry 3, *r1* to IRF entry 1, and *r2* to IRF entry 2. After the IRF registers are loaded, the NFU execution is started with the **EXEC_NFU** instruction. Once the NFU reports it is done (into the main pipeline control), the output values can be retrieved from the NFU using **POP_NFU** instructions (in the order *o1* and then *o2*). Here we've illustrated by placing them immediately after the **EXEC_NFU** though their placement could be scheduled later.

```
1  r4 = fsub r17, r9
2  SET_LOAD_NFU #NFU0, 3, 1, 0, 2
3  r3 = fmul r11, r21 # Input i3 of FU
4  r1 = fsub r7, r5   # Input i1 of FU
5  r9 = fadd r11, r8
6  r2 = fmul r13, r15 # Input i2 of FU
7  EXEC_NFU #NFU0, 1
8  POP_NFU R30, #NFU0
9  POP_NFU R31, #NFU0
```

Listing 3: Modified code utilizing NFU of Listing 1

5.3 Accelerator Reuse

The NOVA toolchain can also support new applications using existing (previously generated) accelerator designs (e.g. running on a system that includes NFUs) by running the *merge* step in reverse to identify paths in the new application that match a path in an already existing NFU. This process starts with the NFU definitions from a previous NOVA run, then merges those accelerators with

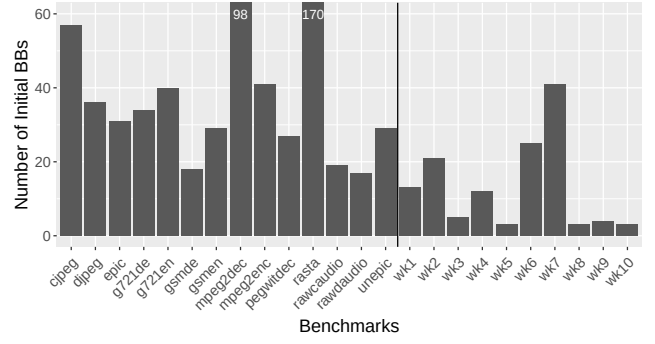


Figure 5: Number of BB candidates to cover at least 95% of total application execution time.

the initial candidates from the new application. If the result yields a complete merge of a path from the new application into an existing NFU definition, then that NFU accelerator can be used to accelerate the computation in the new workload, and the compiler can emit code to utilize that NFU.

5.4 Overheads

Invoking an NFU involves relatively little performance overhead, embodied in the live-in prefix instructions and the popping of results back to core registers. The overheads are therefore directly related to the number of NFU inputs and outputs, which can be controlled during accelerator selection.

The live-in values are forwarded to the NFU as their generator instructions write-back (normal RISC-V register write) in the normal pipeline, and at worst require one prefix instruction each. Recall that the prefix instructions only modify the decoding of some following instructions, allowing execution of **SET_LOAD_NFU** to be implemented as part of the decode processing, limiting the overhead to the fetch and decode of that instruction. The invocation of the NFU is a single instruction, while the write-back of each output value requires a **POP_NFU** instruction, which also ensures that the output value is guaranteed a "write-back" slot in the main pipeline.

6 EXPERIMENTAL METHODOLOGY

We have implemented NOVA framework within the LLVM 12.0 environment – the latest LLVM when we started implementation. While the NOVA framework and methodology is ISA agnostic, the profiled information was gathered using an x86 Linux machine. For our experimental evaluation, we apply NOVA to the Mediabench [42] and EEMBC AutoBench 2.0 workloads [17]. The Mediabench workloads contain a mix of video, audio and encryption coding and decoding benchmarks, while the EEMBC AutoBench represent common workloads for the automotive domain where the individual benchmarks are grouped to create 10 workloads: bitmnp-rspeed-puwmod (wk1), rspeed-idctrn-canrdr (wk2), ttsprk-a2time-pntrch-aifrf (wk3), rspeed-idctrn-iirflt (wk4), ttsprk-a2time-pntrch-idctrn (wk5), matrix-tblook (wk6), ttsprk-a2time-matrix (wk7), ttsprk-a2time-pntrch-tblook (wk8), puwmod-rspeed (wk9), ttsprk-a2time-pntrch (wk10).

NOVA analysis outputs a set of accelerator candidates that covers *all* of the original candidate IS, either individually, or as part

of a new, merged IS. The tool can also be configured to provide stand-alone accelerator candidates with no overlap. Our focus is on high-area-efficiency accelerators, so here we focus only on those accelerator candidates that include merges of at least two original candidate IS. Note that in this case, a benchmark for which we report no candidate accelerators indicates only that advantageous merged accelerator candidates were not found, often because the set of initial candidates was small and didn't provide many merging opportunities.

The evaluation functions described in Section 3 are calibrated using an automatic profiling process with LLVM and an HLS tool [11, 63]. First, an LLVM program generates the target LLVM instructions to be profiled. We then transform the LLVM bitcode into OpenCL² and the resulting program is then fed to an HLS toolchain [63] to generate a hardware implementation of each instruction, which provides the hardware implementation's delay and area statistics for each of the targeted LLVM instructions.

Finally, to obtain the candidate accelerator's area overhead and speedup, we transform the full accelerator's LLVM bitcode to OpenCL and use that to drive the HLS tool [63] to produce a hardware design, from which we can extract area and delay costs, and compare against possible core implementations. Our work focuses on the development of customized embedded cores in heterogeneous SoCs, and here we illustrated NOVIA's capabilities using the RISC-V open-source hardware [61], but NOVIA is broadly applicable to any other embedded core. Specifically, we compare the hardware costs of our proposed accelerators in relation to a RISC-V Ariane core which we have implemented in an SoC design, with its memory and NoC interface, and tested on an FPGA. Our RISC-V Ariane core tile, with its 32K L2 cache, shows on average an area occupancy of 78,609 LUTs. Note that our design is smaller than that in [6], but ours does not include the L1.5 cache, and has a smaller L2 cache. If we were to include these larger elements, the relative area overhead of inline accelerators would be even smaller.

7 RESULTS

We start by applying the methodology to each of the Mediabench benchmarks and EEMBC workloads individually, profiling the benchmark and selecting sufficient BBs to cover at least 95% of the overall execution time. In Figure 5 we show the number of initial BB candidates required to cover 95% execution time per benchmark.

NOVIA applies the merge process of Section 3.2.1 to the candidates, and then the split process of Section 3.3 to extract the final accelerator candidates. Figure 6 shows the box-and-whisker plots of area overhead and the speedup of the final accelerator candidates for each benchmark. Each dot represents a candidate accelerator. We observe that NOVIA is able to extract accelerators that can provide substantial speedup with minimal area overhead for almost all benchmarks. Considered over the full collection of merged accelerators identified by NOVIA, the geometric mean of aggregated speedup that can be provided per benchmark with an area overhead below 7% is 1.42x. Across the pool of accelerators, 75% of them

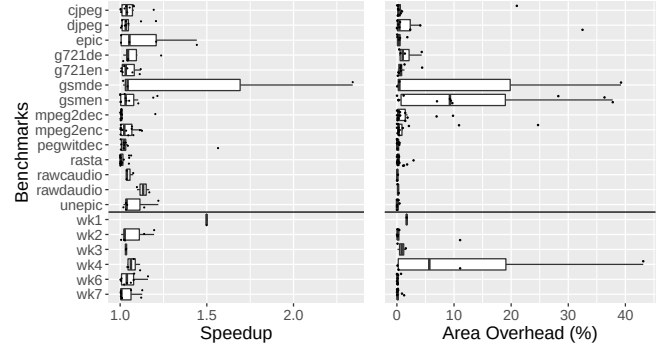


Figure 6: Summary of the per candidate (left) speedup and (right) area overhead (%) for each benchmark. Each dot represents an accelerator candidate.

exhibit less than 1% area overhead. Of those accelerators, 25% of them offer speedup improvement above 1.05x.

To examine the overall performance potential that NOVIA accelerators can have on a full workload, even at small total area overhead, Figure 7 shows aggregate workload speedup attained by implementing a set of NOVIA's proposed inline accelerators, for three different area growth limits on the overall core area. For example, *epic*, with several inline accelerators added to the core, can achieve an aggregate 2.15x workload speedup at 3% total core area overhead. For at least 7 of our benchmarks speedups are consistently high, exceeding 1.5x even with low area overheads, and for the rest substantial improvements can be provided with additional budget area. Note that NOVIA was not able to find worthy merged accelerator candidates for the EEMBC benchmarks *wk5*, *wk8*, *wk9*, *wk10* due to the low number of initial candidates (see Figure 5). This however does not mean that there is no potential for acceleration in those workloads, only that NOVIA was unable to find highly area efficient merged accelerators that provided significant additional benefit.

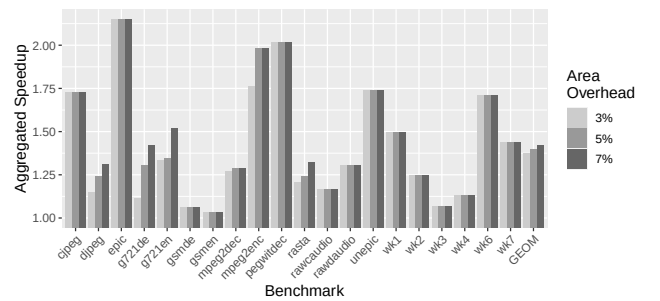


Figure 7: Aggregated total application speedup when analyzed with different area overhead budgets.

To illustrate an inline accelerator candidate, Figure 8 shows the NOVIA DFG for one simple candidate, that parallelizes portions of two different BBs from the *idctm* (inverse discrete cosine transform) benchmark, and provides an aggregate speedup of 1.2x on the workload. In this example, the compiler unrolled inner loops, providing eight parallel multiply-add operations, and NOVIA was

²OpenCL is used as an intermediate step; our tools don't directly convert LLVM IR to HDL. Instead, a proprietary tool parses LLVM IR code and generates compliant OpenCL, which can directly feed our HLS tools.

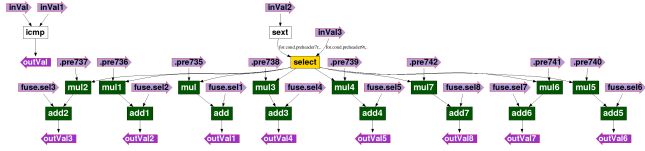


Figure 8: Example proposed inline accelerator for application *wk2*, partially accelerating two different BBs from *idc-trn*, improving application performance by 1.2x for an area cost of about 1%.

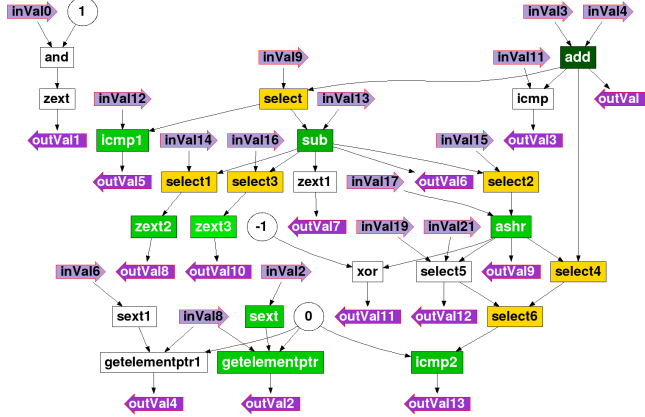


Figure 9: Example of proposed inline accelerator for application *cjpeg*, partially accelerating 17 different BBs, improving application performance by 1.08x for an area cost below 1%. Right facing arrows are inputs, and left facing arrows are outputs. Green boxes indicate "merged" operations used in the original BB executions.

able to detect and merge both BBs. This shows NOVIA can detect SIMDizable code and provide a candidate accelerator for that operation set.

While the DFG of Figure 8 is easy to understand, and the underlying speedup is really just exploiting SIMD operations (though here very efficiently targeted to the specific uses in the workload), most of the accelerator candidates are more complex. Figure 9 displays a less conventional accelerator candidate, extracted for the *djpeg* benchmark, and estimated to provide 1.08x speedup for 1% area overhead. This accelerator merges parts of 17 BBs from various functions of the progressive Huffman coding-compression algorithm.

7.1 Area Efficiency of Multi-Path Merging

One of the distinguishing features of NOVIA is the merge/split process that coalesces common subgraph computations into common hardware, that results in more area-efficient accelerator designs. Figure 10 illustrates the value of this approach. In Figure 10, each final, merged accelerator candidate is compared to the projected area required to implement independent, non-merged accelerators for the same functionality. On average, NOVIA's merged accelerators require about 42% less area than the separate, individual-path inline accelerators, with some accelerators reaching closer to 80% less area.

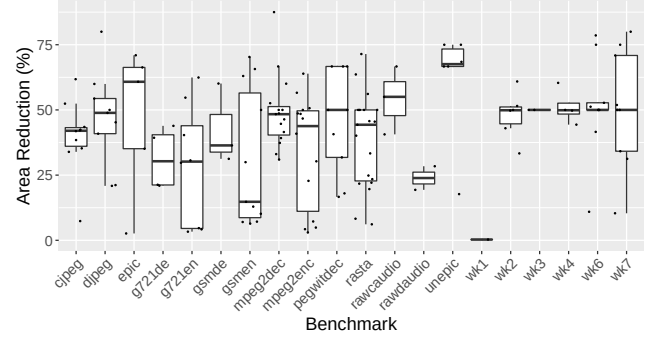


Figure 10: Summary of area reduction of merged NOVIA accelerator versus individual accelerators per path. Each dot represents an accelerator candidate.

	Execution Time	Initial Candidates	Final Candidates	Speedup (3% Δ)	Speedup (5% Δ)	Speedup (7% Δ)
Media	85%	69	19	1.30x	1.30x	1.30x
	90%	85	21	1.30x	1.32x	1.32x
	95%	130	31	1.33x	1.33x	1.33x
Media	85%	326	65	1.33x	1.34x	1.35x
	90%	426	84	1.37x	1.41x	1.43x
	95%	646	118	1.39x	1.44x	1.46x

Table 1: Sensitivity Analysis of number of Initial IS Candidates on aggregate speedup for each benchmark suite, across different area overhead budgets (Δ). Larger initial candidate sets generally increase potential speedup.

7.2 Different Numbers of Initial Candidates

Recall that NOVIA starts with an initial pool of candidate IS (in this case BBs) that cover the majority of the workload execution time. Table 1 summarizes a sensitivity analysis conducted across different sets of initial candidate BBs, where we display aggregated results across all benchmarks in each suite. Each row represents starting NOVIA with a (sorted) set of BBs that cover a specified percentage of the total workload execution time. In each case, NOVIA then runs and produces the full set of merged accelerator candidates from that input set. Table 1 shows for each initial candidate set the number of final accelerator candidates and the aggregate workload speedup they represent for different core area overheads. As a general rule, increasing the number of initial candidates also increases the available speedup, as NOVIA is able to find better merges that result in better total execution coverage. Figure 11 shows the speedup versus area overhead per each Mediabench benchmark. We zoomed in to the low area overhead section where most of our candidates concentrate. For some of the benchmarks, the curve is flat (e.g., *rawaudio*, *unepic*). In those cases, few or single candidates exist, therefore increasing the allocated area does not provide additional benefits. Others display a gradually increasing speedup (e.g., *djpeg*) where there is a variety of available accelerators. Finally, some candidates show large step increases (e.g., *gsmde*) indicating a small number of available candidates, but with wide variations in terms of area cost and speedup potential.

7.3 Generality of NOVIA

While our research is focused on embedded, heterogeneous SoCs, NOVIA is capable of exploring and analyzing any workload that

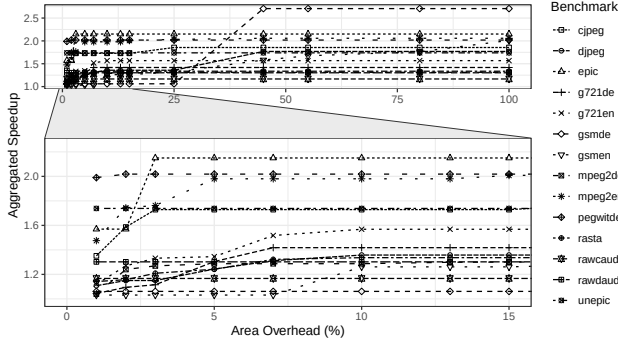


Figure 11: Mediabench aggregated speedup versus area overhead when starting NOVIA with an initial set of BBs that covers 95% of the total workload execution time.

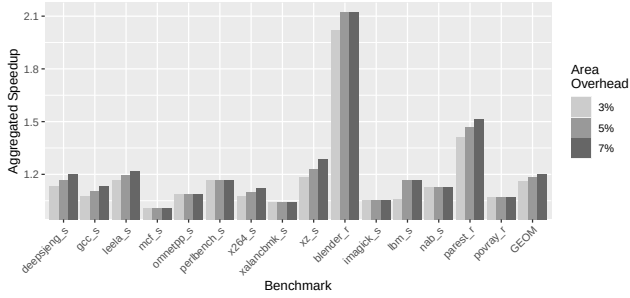


Figure 12: Aggregated speedups achieved for the SPEC C/C++ benchmarks using different area overhead budgets.

can be represented in the LLVM IR. To demonstrate this with a familiar workload set, we also applied NOVIA analysis to the C/C++ SPEC benchmarks. Figure 12 illustrates the aggregate benchmark speedup obtained for different area overhead budgets across the C/C++ SPEC benchmarks. Note that NOVIA does not report accelerator candidates for *namd_r* due to a low number of initial BBs and high concentration of memory operations in them, severely constraining merge opportunities.

SPEC workloads tend to represent a worst-case scenario for our tools, with the majority of benchmarks containing a high number of small basic blocks, more distributed computation across more complex workload flows, and a high prevalence of memory operations interspersed within the computation. These interspersed memory operations present a particular complexity for our inline accelerators, as we do not want to add new memory-access paths from the accelerators into the core pipeline. These irregular computations therefore limit the ability to merge paths and merge BB operations to form highly efficient accelerators, but even so NOVIA still finds accelerators that can efficiently provide significant core-level speedups of up to 30% and 40% with just a small 5% core area overhead.

7.4 Multi-Workload Evaluation

NOVIA also allows analysis of a workload suite consisting of multiple applications, and searching across these for accelerator candidates that most improve overall workload statistics. NOVIA can

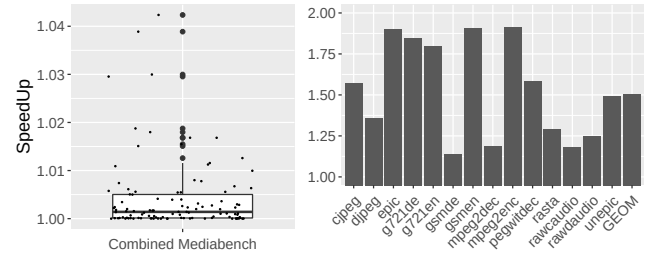


Figure 13: Multi-Workload evaluation of Mediabench (left) Speedup for each of the final candidates of the multi-workload analysis. (right) individual benchmark speedup obtained from the multi-workload evaluation.

analyze such a workload suite by first gathering the individual workload profiles, and then adjusting them for global suite analysis by applying per-application, user-defined weights to reflect the relative execution mix for the individual workloads in the suite. The individual application bitcode files are also sent through a renaming transformation (to avoid BB and function name collisions in the LLVM IR) and the renamed bitcode files are linked together into a single unified full-suite bitcode file. Here, we applied NOVIA to Mediabench, but as a single multi-workload suite. Starting with the initial BBs from the previous individual workload analysis, we get 646 initial candidates. NOVIA then proceeds in the same manner as per a single application evaluation, applying the merge process for the full workload suite, and then the split process to extract the final accelerator candidates targeting multiple applications. Figure 13(left) shows the speed-up for each of the extracted accelerators for the Mediabench workload, that cover the entire Mediabench suite. Although implementing all the accelerators might be infeasible, designers could select any subset (e.g. the top fliers, or one that best fits an area budget). Implementing all of the proposed accelerators (i.e. no area constraint) is estimated to provide a 1.82x Mediabench speedup. In Figure 13 (right) we show for each component benchmark the speedup obtained by that full set of 106 multi-workload suite inline accelerators.

8 RELATED WORK

Workload analysis is an area with a rich history [7, 16, 30, 45]. Early works considered the granularity at which analysis (and optimization) should be done, including Superblocks [30] and Hyperblocks [45]. Superblock and Hyperblock formation both required tuned heuristics [4, 5]. VLIW processors' compilers use coarse-grained region formations by exploiting hardware predication [1, 44, 45]. Other work [43] constructed larger granularity offloads by leveraging branch predictor. There are also works that explore the most efficient means to extract the profiling information, culminating in the Ball-Larus techniques [7]. Several other work have proposed to construct synthetic benchmarks based on the workload analysis [9, 10, 19, 31–37].

There are also a number of previous works that consider the selection of the targeted accelerators. In [65] the authors employ a technique that grows an accelerator from a set of DFG-linked basic blocks, trying to form a sufficiently large accelerator body to

provide speedup overall, while maintaining a single path of functionality. In a subsequent work [64], the authors take a similar approach to [65] but adjust the scale by using workload-defined functions as their unit of analysis. This provides a larger accelerator definition by combining functions of the workload into an accelerator body, and can produce good offload accelerator candidates. Similar tools experiment around the definition of their target accelerators to provide useful and diverse insights on the right accelerator granularity like finding hot chained dependencies [40, 49, 58] or estimating their potential accelerability [38, 52, 54]. Needle [39] attempts to provide a methodology and toolset that implements a series of LLVM passes to analyze the whole-program LLVM IR. By using a Ball-Larus profiling [7], Needle identifies multi-basic block paths that constitute contiguous functionality, which they convert into offload (source-code) functions. These functions can then be used to derive accelerators. [27, 29] starts from a similar granularity definition but targets energy-efficient custom architectures. Similarly, Conservation Cores and GreenDroid [24, 59] take a different approach, by proposing customized hardware focusing on energy efficiency, sometimes at the expense of speedup, without applying any merge/split techniques. The successor, QSCores [60], heavily focuses on larger, programmable off-load accelerator cores connected to a shared memory interface. QSCores modify the core to use the scan chain to communicate to the QSCores, and add some ISA elements to the core to enact such communications. The use of scan chains to communicate to and invoke the QSCores is interesting and novel, but is not a high-performance, high-bandwidth, low-overhead interface which therefore requires QSCores to implement and off-load longer-running workload computations. NOVA targets inline accelerators that operate as (single-instruction) functional units (FUs) within the main core pipeline, which stresses different aspects of the design and integration, and provides very different constraints on the accelerator (FU) implementation. NOVA also applies a novel splitting process that allows the tool to target accelerators that meet specific design constraints. This is particularly valuable for inline accelerators, which require relatively low added core area overhead, highly constrained numbers of live-ins and live-outs, and execution times on the order of one or a few cycles (to help retain the core's native ILP exploitation). This focus on low-overhead inline accelerators prohibits the inclusion/replication of large design elements such as a load-store unit (which can be included in off-load engines like QSCores) precluding our accelerators from independent memory access. NOVA's acceleration is therefore applicable even in smaller, lighter weight systems that do not provide as complete or advanced shared-memory coherence networks and SMP support. In this work, NOVA is used to target acceleration (i.e., workload performance) under area budget constraints, while QSCores targets energy efficiency. Both are important targets, but the resulting approaches and targeted accelerator designs can differ significantly, especially in the details of accelerator selection and implementation. Note, however, that these are not incompatible approaches; one could implement NOVA accelerators in the core, and combine that core with any form of off-load acceleration, including QSCores.

In terms of accelerator realization, there is work in the analysis of basic blocks to find common computational subgraphs, starting with common sub-expression elimination compiler optimization. In [53],

the authors focus on the software, and try to identify similar code segments, with the goal of reducing the code size at the function level. Some works explore the possibilities of finding acceleratable regions by focusing on developing algorithms to generate optimal or close to optimal cuts and subgraphs to realize feasible accelerators depending on the diverse constraints and requirements [2, 3, 50].

There has also been some prior work that propose methods to discover useful new instructions. Works like [21–23, 48] use novel computational representation to propose and find efficient accelerators, however, those do not consider covering multiple computational paths (e.g., by functionality merging) in an inline accelerator body. There is also a body of work focusing on the implementation of a substrate for the proposed accelerators or architectures [18]. DSAGEN, DySER and Tartan [8, 25, 26, 28, 46, 62] propose reconfigurable architectures with compiler support (through predication) that is able to identify and exploit data-level parallelism. Custom-Fit [18] instead works under the VLIW architecture paradigm. Those kinds of architectures can fail to provide improvements when meeting the constraints of an embedded design. Further, they lack the end-to-end methodology of consolidating the instruction sequences, and splitting to generate representative subgraphs for inline acceleration. Other approaches argue for the intensive use of small self-contained domain-specific accelerators (Sea of Accelerators) and corresponding simulators to do an early evaluation [55–57]. Finally, to address accelerator reuse, some works target the problem of mapping custom instructions to arbitrary workloads. Authors also couple those techniques with minimal integration effort generalized accelerators in order to facilitate integration and reusability [12–15, 20, 47].

Overall, none of the prior art tackles the full problem of efficient inline acceleration of arbitrary granularity with an end-to-end methodology. NOVA provides a framework that starts with a workload (source code), automatically analyzes the workload to identify significant work segments, selects a set of these and explores optimizations of them as hardware accelerators within a core.

9 CONCLUSIONS

In this paper, we have presented NOVA, a framework to derive inline accelerators from an analysis of workload code. NOVA applies workload profiling and a merge/split process to coalesce common sub-graph computations to derive area-efficient accelerator designs. We demonstrate NOVA capability for a RISC-V core. NOVA discovers many acceleration opportunities, that improve performance of various benchmark suites like EEMBC Autobench 2.0 and Mediabench by 1.37x on average with only a 3% core area increase.

ACKNOWLEDGMENTS

This research was developed with funding from the Defense Advanced Research Projects Agency (DARPA). The views, opinions and/or other findings expressed are those of the authors and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government. This document is approved for public release: distribution unlimited.

REFERENCES

- [1] J. R. Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. 1983. Conversion of Control Dependence to Data Dependence. In *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '83)*. Association for Computing Machinery, New York, NY, USA, 177–189. <https://doi.org/10.1145/567067.567085>
- [2] Kubilay Atasu, Can C. Özturan, Günhan Dündar, Oskar Mencer, and Wayne Luk. 2008. CHIPS: Custom Hardware Instruction Processor Synthesis. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 27, 3 (2008), 528–541. <https://doi.org/10.1109/TCAD.2008.915536>
- [3] Kubilay Atasu, Laura Pozzi, and Paolo Ienne. 2003. Automatic application-specific instruction-set extensions under microarchitectural constraints. In *Proceedings of the 40th Design Automation Conference, DAC 2003, Anaheim, CA, USA, June 2–6, 2003*. ACM, 256–261. <https://doi.org/10.1145/775832.775897>
- [4] David I. August, Daniel A. Connors, Scott A. Mahlke, John W. Sias, Kevin M. Crozier, Ben-Chung Cheng, Patrick R. Eaton, Qudus B. Olaniran, and Wen-mei W. Hwu. 1998. Integrated Predicated and Speculative Execution in the IMPACT EPIC Architecture. In *Proceedings of the 25th Annual International Symposium on Computer Architecture, ISCA 1998, Barcelona, Spain, June 27 - July 1, 1998*, Mateo Valero, Gurindar S. Sohi, and Doug DeGroot (Eds.). IEEE Computer Society, 227–237. <https://doi.org/10.1109/ISCA.1998.694777>
- [5] David I. August, Wen-mei W. Hwu, and Scott A. Mahlke. 1997. A Framework for Balancing Control Flow and Predication. In *Proceedings of the Thirtieth Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 30, Research Triangle Park, North Carolina, USA, December 1–3, 1997*, Mark Smotherman and Tom Conte (Eds.). ACM/IEEE Computer Society, 92–103. <https://doi.org/10.1109/MICRO.1997.645801>
- [6] Jonathan Balkind, Michael McKeown, Yaosheng Fu, Tri Minh Nguyen, Yanqi Zhou, Alexey Lavrov, Mohammad Shahrad, Adi Fuchs, Samuel Payne, Xiaohua Liang, Matthew Matl, and David Wentzlaff. 2016. OpenPiton: An Open Source Manycore Research Framework. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '16, Atlanta, GA, USA, April 2–6, 2016*, Tom Conte and Yuanquan Zhou (Eds.). ACM, 217–232. <https://doi.org/10.1145/2872362.2872414>
- [7] Thomas Ball and James R. Larus. 1996. Efficient Path Profiling. In *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 29, Paris, France, December 2–4, 1996*, Stephen W. Melvin and Steve Beatty (Eds.). ACM/IEEE Computer Society, 46–57. <https://doi.org/10.1109/MICRO.1996.566449>
- [8] Jesse Benson, Ryan Cofell, Chris Frericks, Chen-Han Ho, Venkatraman Govindaraju, Tony Nowatzki, and Karthikeyan Sankaralingam. 2012. Design, integration and implementation of the DySER hardware accelerator into OpenSPARC. In *18th IEEE International Symposium on High Performance Computer Architecture, HPCA 2012, New Orleans, LA, USA, 25–29 February, 2012*. IEEE Computer Society, 115–126. <https://doi.org/10.1109/HPCA.2012.6168949>
- [9] Ramon Bertran, Alper Buyuktosunoglu, Meeta Sharma Gupta, Marc González, and Pradip Bose. 2012. Systematic Energy Characterization of CMP/SMT Processor Systems via Automated Micro-Benchmarks. In *45th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2012, Vancouver, BC, Canada, December 1–5, 2012*. IEEE Computer Society, 199–211. <https://doi.org/10.1109/MICRO.2012.27>
- [10] Calvin Bulla, Lluç Alvarez, Miquel Moretó, Ramon Bertran, Alper Buyuktosunoglu, and Pradip Bose. 2018. ChopStix: Systematic Extraction of Code-Representative Microbenchmarks. In *2018 IEEE International Symposium on Workload Characterization, IISWC 2018, Raleigh, NC, USA, September 30 - October 2, 2018*. IEEE Computer Society, 80–81. <https://doi.org/10.1109/IISWC.2018.8573473>
- [11] Andrew Canis, Jongsok Choi, Mark Aldham, Victor Zhang, Ahmed Kammoona, Jason Helge Anderson, Stephen Dean Brown, and Tomasz S. Czajkowski. 2011. LegUp: high-level synthesis for FPGA-based processor/accelerator systems. In *Proceedings of the ACM/SIGDA 19th International Symposium on Field Programmable Gate Arrays, FPGA 2011, Monterey, California, USA, February 27, March 1, 2011*, John Wawrzyniak and Katherine Compton (Eds.). ACM, 33–36. <https://doi.org/10.1145/1950413.1950423>
- [12] Nathan Clark, Jason A. Blome, Michael L. Chu, Scott A. Mahlke, Stuart Biles, and Krisztián Flautner. 2005. An Architecture Framework for Transparent Instruction Set Customization in Embedded Processors. In *32nd International Symposium on Computer Architecture (ISCA 2005)*, 4–8 June 2005, Madison, Wisconsin, USA. IEEE Computer Society, 272–283. <https://doi.org/10.1109/ISCA.2005.9>
- [13] Nathan Clark, Manjunath Kudlur, Hyunchul Park, Scott A. Mahlke, and Krisztián Flautner. 2004. Application-Specific Processing on a General-Purpose Core via Transparent Instruction Set Customization. In *37th Annual International Symposium on Microarchitecture (MICRO-37 2004)*, 4–8 December 2004, Portland, OR, USA. IEEE Computer Society, 30–40. <https://doi.org/10.1109/MICRO.2004.5>
- [14] Nathan Clark, Hongtao Zhong, and Scott A. Mahlke. 2003. Processor Acceleration Through Automated Instruction Set Customization. In *Proceedings of the 36th Annual International Symposium on Microarchitecture, San Diego, CA, USA, December 3–5, 2003*. IEEE Computer Society, 129–140. <https://doi.org/10.1109/MICRO.2003.1253189>
- [15] Nathan Clark, Hongtao Zhong, and Scott A. Mahlke. 2005. Automated Custom Instruction Generation for Domain-Specific Processor Acceleration. *IEEE Trans. Computers* 54, 10 (2005), 1258–1270. <https://doi.org/10.1109/TC.2005.156>
- [16] Lieven Eeckhout, Robert H. Bell Jr., Bastiaan Stougie, Koen De Bosschere, and Lizy Kurian John. 2004. Control Flow Modeling in Statistical Simulation for Accurate and Efficient Processor Design Studies. In *31st International Symposium on Computer Architecture (ISCA 2004)*, 19–23 June 2004, Munich, Germany. IEEE Computer Society, 350–363. <https://doi.org/10.1109/ISCA.2004.1310787>
- [17] EEMBC Consortium. [n. d.]. EEMBC Autobench 2.0 Website. <https://www.eembc.org/autobench/>
- [18] Joseph A. Fisher, Paolo Faraboschi, and Giuseppe Desoli. 1996. Custom-fit Processors: Letting Applications Define Architectures. In *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 29, Paris, France, December 2–4, 1996*, Stephen W. Melvin and Steve Beatty (Eds.). ACM/IEEE Computer Society, 324–335. <https://doi.org/10.1109/MICRO.1996.566472>
- [19] Karthik Ganesan, Jungho Jo, and Lizy K. John. 2010. Synthesizing memory-level parallelism aware miniature clones for SPEC CPU2006 and ImplantBench workloads. In *IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2010, 28–30 March 2010, White Plains, NY, USA*. IEEE Computer Society, 33–44. <https://doi.org/10.1109/ISPASS.2010.5452076>
- [20] Ricardo E. Gonzalez. 2000. Xtensa: A Configurable and Extensible Processor. *IEEE Micro* 20, 2 (2000), 60–70. <https://doi.org/10.1109/40.848473>
- [21] Cecilia González-Alvarez, Jennifer B. Sartor, Carlos Álvarez, Daniel Jiménez-González, and Lieven Eeckhout. 2013. Accelerating an application domain with specialized functional units. *ACM Trans. Archit. Code Optim.* 10, 4 (2013), 47:1–47:25. <https://doi.org/10.1145/2541228.2555303>
- [22] Cecilia González-Alvarez, Jennifer B. Sartor, Carlos Álvarez, Daniel Jiménez-González, and Lieven Eeckhout. 2015. Automatic design of domain-specific instructions for low-power processors. In *26th IEEE International Conference on Application-specific Systems, Architectures and Processors, ASAP 2015, Toronto, ON, Canada, July 27–29, 2015*. IEEE Computer Society, 1–8. <https://doi.org/10.1109/ASAP.2015.7245697>
- [23] Cecilia González-Alvarez, Jennifer B. Sartor, Carlos Álvarez, Daniel Jiménez-González, and Lieven Eeckhout. 2016. MnGLE: An Efficient Framework for Domain Acceleration Using Low-Power Specialized Functional Units. *ACM Trans. Archit. Code Optim.* 13, 2 (2016), 17:1–17:26. <https://doi.org/10.1145/2898356>
- [24] Nathan Goulding-Hotta, Jack Sampson, Ganesh Venkatesh, Saturnino Garcia, Joe Auricchio, Po-Chao Huang, Manish Arora, Siddhartha Nath, Vikram Bhatt, Jonathan Babb, Steven Swanson, and Michael Bedford Taylor. 2011. The Green-Droid Mobile Application Processor: An Architecture for Silicon's Dark Future. *IEEE Micro* 31, 2 (2011), 86–95. <https://doi.org/10.1109/MM.2011.18>
- [25] Venkatraman Govindaraju, Chen-Han Ho, Tony Nowatzki, Jatin Chhugani, Nadathur Satish, Karthikeyan Sankaralingam, and Changkyu Kim. 2012. DySER: Unifying Functionality and Parallelism Specialization for Energy-Efficient Computing. *IEEE Micro* 32, 5 (2012), 38–51. <https://doi.org/10.1109/MM.2012.51>
- [26] Venkatraman Govindaraju, Chen-Han Ho, and Karthikeyan Sankaralingam. 2011. Dynamically Specialized Datapaths for energy efficient computing. In *17th International Conference on High-Performance Computer Architecture (HPCA-17 2011)*, February 12–16 2011, San Antonio, Texas, USA. IEEE Computer Society, 503–514. <https://doi.org/10.1109/HPCA.2011.5749755>
- [27] Shantanu Gupta, Shuguang Feng, Amin Ansari, Scott A. Mahlke, and David I. August. 2011. Bundled execution of recurring traces for energy-efficient general purpose processing. In *44rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2011, Porto Alegre, Brazil, December 3–7, 2011*, Carlo Galuzzi, Luigi Carro, Andreas Moshovos, and Milos Prvulovic (Eds.). ACM, 12–23. <https://doi.org/10.1145/2155620.2155623>
- [28] Chen-Han Ho, Venkatraman Govindaraju, Tony Nowatzki, Ranjini Nagaraju, Zachary Marzec, Preeti Agarwal, Chris Frericks, Ryan Cofell, and Karthikeyan Sankaralingam. 2015. Performance evaluation of a DySER FPGA prototype system spanning the compiler, microarchitecture, and hardware implementation. In *2015 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2015, Philadelphia, PA, USA, March 29–31, 2015*. IEEE Computer Society, 203–214. <https://doi.org/10.1109/ISPASS.2015.7095806>
- [29] Justin Holewinski, Ragavendar Ramamurthi, Mahesh Ravishanker, Naznin Fausia, Louis-Noël Pouchet, Atanas Rountev, and P. Sadayappan. 2012. Dynamic trace-based analysis of vectorization potential of applications. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*, Jan Vitek, Haibo Lin, and Frank Tip (Eds.). ACM, 371–382. <https://doi.org/10.1145/2254064.2254108>
- [30] Wen-mei W. Hwu, Scott A. Mahlke, William Y. Chen, Pohua P. Chang, Nancy J. Warter, Roger A. Bringmann, Roland G. Ouellette, Richard E. Hank, Tokuzo Kiyohara, Grant E. Haab, John G. Holm, and Daniel M. Lavery. 1993. The superblock: An effective technique for VLIW and superscalar compilation. *J. Supercomput.* 7, 1–2 (1993), 229–248. <https://doi.org/10.1007/BF01205185>
- [31] Lizy Kurian John. 2011. Proprietary code to non-proprietary benchmarks: synthesis techniques for scalable benchmarks. In *ICPE '11 - Second Joint WOSP/SIPEW International Conference on Performance Engineering, Karlsruhe, Germany, March*

- 14-16, 2011, Samuel Kounev, Vittorio Cortellessa, Raffaella Mirandola, and David J. Lilja (Eds.). ACM, 1–2. <https://doi.org/10.1145/1958746.1958748>
- [32] Ajay Joshi, Lieven Eeckhout, Robert H. Bell Jr., and Lizy Kurian John. 2006. Performance Cloning: A Technique for Disseminating Proprietary Applications as Benchmarks. In *Proceedings of the 2006 IEEE International Symposium on Workload Characterization, IISWC 2006, October 25-27, 2006, San Jose, California, USA*. IEEE Computer Society, 105–115. <https://doi.org/10.1109/IISWC.2006.302734>
- [33] Ajay Joshi, Lieven Eeckhout, Robert H. Bell Jr., and Lizy Kurian John. 2008. Distilling the essence of proprietary workloads into miniature benchmarks. *ACM Trans. Archit. Code Optim.* 5, 2 (2008), 10:1–10:33. <https://doi.org/10.1145/1400112.1400115>
- [34] Ajay M. Joshi, Lieven Eeckhout, and Lizy Kurian John. 2007. Exploring the Application Behavior Space Using Parameterized Synthetic Benchmarks. In *16th International Conference on Parallel Architectures and Compilation Techniques (PACT 2007), Brasov, Romania, September 15-19, 2007*. IEEE Computer Society, 412. <https://doi.org/10.1109/PACT.2007.31>
- [35] Ajay M. Joshi, Lieven Eeckhout, Lizy Kurian John, and Ciji Isen. 2008. Automated microprocessor stressmark generation. In *14th International Conference on High-Performance Computer Architecture (HPCA-14 2008), 16-20 February 2008, Salt Lake City, UT, USA*. IEEE Computer Society, 229–239. <https://doi.org/10.1109/HPCA.2008.4658642>
- [36] Robert H. Bell Jr., Rajiv R. Bhatia, Lizy K. John, Jeff Stuecheli, John Griswell, Paul Tu, Louis Capps, Anton Blanchard, and Ravel Thai. 2006. Automatic testcase synthesis and performance model validation for high performance PowerPC processors. In *2006 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2006, March 19-21, 2006, Austin, Texas, USA, Proceedings*. IEEE Computer Society, 154–165. <https://doi.org/10.1109/ISPASS.2006.1620800>
- [37] Robert H. Bell Jr. and Lizy Kurian John. 2005. Improved automatic testcase synthesis for performance model validation. In *Proceedings of the 19th Annual International Conference on Supercomputing, ICS 2005, Cambridge, Massachusetts, USA, June 20-22, 2005, Arvind and Larry Rudolph (Eds.)*. ACM, 111–120. <https://doi.org/10.1145/1088149.1088164>
- [38] Snehasish Kumar, Vijayalakshmi Srinivasan, Amiral Sharifian, Nick Sumner, and Arrvinth Shriraman. 2016. Peruse and Profit: Estimating the Accelerability of Loops. In *Proceedings of the 2016 International Conference on Supercomputing, ICS 2016, Istanbul, Turkey, June 1-3, 2016, Ozcan Ozturk, Kemal Ebcioğlu, Mahmut T. Kandemir, and Onur Mutlu (Eds.)*. ACM, 21:1–21:13. <https://doi.org/10.1145/2925426.2926269>
- [39] Snehasish Kumar, Nick Sumner, Vijayalakshmi Srinivasan, Steve Margerm, and Arrvinth Shriraman. 2017. Needle: Leveraging Program Analysis to Analyze and Extract Accelerators from Whole Programs. In *2017 IEEE International Symposium on High Performance Computer Architecture, HPCA 2017, Austin, TX, USA, February 4-8, 2017*. IEEE Computer Society, 565–576. <https://doi.org/10.1109/HPCA.2017.59>
- [40] Snehasish Kumar, William N. Sumner, and Arrvinth Shriraman. 2016. SPEC-AX and PARSEC-AX: extracting accelerator benchmarks from microprocessor benchmarks. In *2016 IEEE International Symposium on Workload Characterization, IISWC 2016, Providence, RI, USA, September 25-27, 2016*. IEEE Computer Society, 117–127. <https://doi.org/10.1109/IISWC.2016.7581272>
- [41] Chris Lattner and Vikram S. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *2nd IEEE / ACM International Symposium on Code Generation and Optimization (CGO 2004), 20-24 March 2004, San Jose, CA, USA*. IEEE Computer Society, 75–88. <https://doi.org/10.1109/CGO.2004.1281665>
- [42] Chunho Lee, Miodrag Potkonjak, and William H. Mangione-Smith. 1997. MediaBench: A Tool for Evaluating and Synthesizing Multimedia and Communications Systems. In *Proceedings of the Thirtieth Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 30, Research Triangle Park, North Carolina, USA, December 1-3, 1997, Mark Smotherman and Tom Conte (Eds.)*. ACM/IEEE Computer Society, 330–335. <https://doi.org/10.1109/MICRO.1997.645830>
- [43] Feng Liu, Heejin Ahn, Stephen R. Beard, Taewook Oh, and David I. August. 2015. DynaSpAM: dynamic spatial architecture mapping using out of order instruction schedules. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture, Portland, OR, USA, June 13-17, 2015, Deborah T. Marr and David H. Albonesi (Eds.)*. ACM, 541–553. <https://doi.org/10.1145/2749469.2750414>
- [44] Scott A. Mahlke, Richard E. Hank, James E. McCormick, David I. August, and Wen-mei W. Hwu. 1995. A Comparison of Full and Partial Predicated Execution Support for ILP Processors. In *Proceedings of the 22nd Annual International Symposium on Computer Architecture, ISCA '95, Santa Margherita Ligure, Italy, June 22-24, 1995, David A. Patterson (Ed.)*. ACM, 138–150. <https://doi.org/10.1145/223982.225965>
- [45] Scott A. Mahlke, David C. Lin, William Y. Chen, Richard E. Hank, and Roger A. Bringmann. 1992. Effective compiler support for predicated execution using the hyperblock. In *Proceedings of the 25th Annual International Symposium on Microarchitecture, Portland, Oregon, USA, November 1992, Wen-mei W. Hwu (Ed.)*. ACM / IEEE Computer Society, 45–54. <https://doi.org/10.1109/MICRO.1992.696999>
- [46] Mahim Mishra, Timothy J. Callahan, Tiberiu Chelcea, Girish Venkataramani, Seth C. Goldstein, and Mihai Budiu. 2006. Tartan: Evaluating Spatial Computation for Whole Program Execution. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)*. Association for Computing Machinery, New York, NY, USA, 163–174. <https://doi.org/10.1145/1168857.1168878>
- [47] P. Mokri and M. Hempstead. 2020. Early-stage Automated Identification Tool for Shared Accelerators. In *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 217–217. <https://doi.org/10.1109/FCCM48280.2020.00048>
- [48] Tony Nowatzki, Venkatraman Govindaraju, and Karthikeyan Sankaralingam. 2015. A Graph-Based Program Representation for Analyzing Hardware Specialization Approaches. *IEEE Comput. Archit. Lett.* 14, 2 (2015), 94–98. <https://doi.org/10.1109/LCA.2015.2476801>
- [49] Angshuman Parashar, Michael Pellauer, Michael Adler, Bushra Ahsan, Neal Clayton Crago, Daniel Lustig, Vladimir Pavlov, Antonia Zhai, Mohit Gambhir, Aamer Jaleel, Randy L. Allmon, Rachid Rayess, Stephen Maresch, and Joel S. Emer. 2013. Triggered instructions: a control paradigm for spatially-programmed architectures. In *The 40th Annual International Symposium on Computer Architecture, ISCA'13, Tel-Aviv, Israel, June 23-27, 2013, Avi Mendelson (Ed.)*. ACM, 142–153. <https://doi.org/10.1145/2485922.2485935>
- [50] Laura Pozzi, Kubilay Atasul, and Paolo Ienne. 2006. Exact and approximate algorithms for the extension of embedded processor instruction sets. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 25, 7 (2006), 1209–1229. <https://doi.org/10.1109/TCAD.2005.855950>
- [51] Tristan Ravitch. 2014. WLLVM : Whole-Program LLVM. <https://github.com/travitch/whole-program-llvm>
- [52] Brandon Reagen, Yakun Sophia Shao, Gu-Yeon Wei, and David M. Brooks. 2013. Quantifying acceleration: Power/performance trade-offs of application kernels in hardware. In *International Symposium on Low Power Electronics and Design (ISLPED), Beijing, China, September 4-6, 2013, Pai H. Chou, Ru Huang, Yuan Xie, and Tanay Karnik (Eds.)*. IEEE, 395–400. <https://doi.org/10.1109/ISLPED.2013.6629329>
- [53] Rodrigo C. O. Rocha, Pavlos Petoumenos, Zheng Wang, Murray Cole, and Hugh Leather. 2019. Function Merging by Sequence Alignment. In *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2019, Washington, DC, USA, February 16-20, 2019, Mahmut Taylan Kandemir, Alexandra Jimborean, and Tipp Moseley (Eds.)*. IEEE, 149–163. <https://doi.org/10.1109/CGO.2019.8661174>
- [54] Yakun Sophia Shao and David M. Brooks. 2013. ISA-independent workload characterization and its implications for specialized architectures. In *2012 IEEE International Symposium on Performance Analysis of Systems & Software, Austin, TX, USA, 21-23 April, 2013*. IEEE Computer Society, 245–255. <https://doi.org/10.1109/ISPASS.2013.6557175>
- [55] Yakun Sophia Shao, Brandon Reagen, Gu-Yeon Wei, and David M. Brooks. 2014. Aladdin: A pre-RTL, power-performance accelerator simulator enabling large design space exploration of customized architectures. In *ACM/IEEE 41st International Symposium on Computer Architecture, ISCA 2014, Minneapolis, MN, USA, June 14-18, 2014*. IEEE Computer Society, 97–108. <https://doi.org/10.1109/ISCA.2014.6853196>
- [56] Yakun Sophia Shao, Brandon Reagen, Gu-Yeon Wei, and David M. Brooks. 2015. The Aladdin Approach to Accelerator Design and Modeling. *IEEE Micro* 35, 3 (2015), 58–70. <https://doi.org/10.1109/MM.2015.50>
- [57] Yakun Sophia Shao, Sam Likun Xi, Vijayalakshmi Srinivasan, Gu-Yeon Wei, and David M. Brooks. 2016. Co-designing accelerators and SoC interfaces using gem5-Aladdin. In *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 48:1–48:12. <https://doi.org/10.1109/MICRO.2016.7783751>
- [58] Amiral Sharifian, Snehasish Kumar, Apala Guha, and Arrvinth Shriraman. 2016. Chainsaw: Von-neumann accelerators to leverage fused instruction chains. In *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 49:1–49:14. <https://doi.org/10.1109/MICRO.2016.7783752>
- [59] Ganesh Venkatesh, Jack Sampson, Nathan Goulding, Saturnino Garcia, Vladyslav Bryksin, Jose Lugo-Martinez, Steven Swanson, and Michael Bedford Taylor. 2010. Conservation cores: reducing the energy of mature computations. (2010), 205–218. <https://doi.org/10.1145/1736020.1736044>
- [60] Ganesh Venkatesh, Jack Sampson, Nathan Goulding-Hotta, Sravanthi Kota Venkata, Michael Bedford Taylor, and Steven Swanson. 2011. QSCORES: Trading dark silicon for scalable energy efficiency with quasi-specific cores. In *2011 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 163–174.
- [61] Andrew Waterman, Yunsup Lee, Rimas Avizienis, Henry Cook, David Patterson, and Krste Asanovic. 2013. The RISC-V instruction set. 1–1. <https://doi.org/10.1109/HOTCHIPS.2013.7478332>
- [62] Jian Weng, Sihao Liu, Vidushi Dadu, Zhengrong Wang, Preyas Shah, and Tony Nowatzki. 2020. DSAGEN: Synthesizing Programmable Spatial Accelerators. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 268–281. <https://doi.org/10.1109/ISCA45697.2020.00032>
- [63] Xilinx. [n. d.]. Vitis High-Level Synthesis. <https://www.xilinx.com/products/design-tools/vivado/integration/esl-design.html>

- [64] Georgios Zacharopoulos, Lorenzo Ferretti, Giovanni Ansaloni, Giuseppe Di Guglielmo, Luca P. Carloni, and Laura Pozzi. 2019. Compiler-Assisted Selection of Hardware Acceleration Candidates from Application Source Code. In *37th IEEE International Conference on Computer Design, ICCD 2019, Abu Dhabi, United Arab Emirates, November 17-20, 2019*. IEEE, 129–137. <https://doi.org/10.1109/ICCD46524.2019.00024>
- [65] Georgios Zacharopoulos, Lorenzo Ferretti, Emanuele Giaquinta, Giovanni Ansaloni, and Laura Pozzi. 2019. RegionSeeker: Automatically Identifying and Selecting Accelerators From Application Source Code. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 38, 4 (2019), 741–754. <https://doi.org/10.1109/TCAD.2018.2818689>